

Open Semantic Interchange

为什么本体需要一个开放标准的指标体系

语义层的孤岛与开放 | 从 **OSI 标准** 到 **Bottom-up** 指标体系

演讲嘉宾：赵恒 @datus.ai

语义层，正在成为 Agent 的“契约语言”

但今天它却在重走 BI 时代的孤岛老路

语义层 = 契约语言

- Agent 与业务、数据库之间的稳定接口
- 同一业务问题 → 同一口径、同一段可追溯 SQL
- 靠的不是更大的模型，而是准确、可治理的：

指标 Metrics

知识库 Knowledge

本体 Ontology

却在重走 BI 孤岛老路

- **Snowflake / Databricks** 各自的 Semantic View / Metric View
- **Looker • Tableau • Superset** BI 厂商纷纷发布语义层
- **Cube • dbt MetricFlow** 第三方独立方案

→ 各有差异，带来新的语义孤岛与生态锁定

结论 契约需要标准。语义层若人人一套，Agent 就永远拿不到可信、可移植的业务口径。

一个 Snowflake 客户，却有四套指标口径

数仓已经统一，语义却四分五裂

Snowflake 数仓

已经比国内分散的
几套开源系统
简单得多——
数据是统一的

✓ 数据层已统一

1

Tableau Semantics

沉淀已久的可视化语义

2

LookML

Looker 里的指标模型

3

StatSig metrics

实验平台里的指标

4

Snowflake Semantic View

数仓原生语义对象

结论 数仓统一 ≠ 指标统一。四套语义各说各话，Agent 该信哪个“收入”？——他需要一套统一的指标预言。

Snowflake 与 Databricks, 今年都在语义层上叠 Ontology

共同的底座: Agent + Semantic Layer

Snowflake

Cortex Agent + Semantic View

- Ontology-grounded Cortex Agents
- KG_NODE / KG_EDGE + Recursive CTE
- 语义层之上叠本体推理

Databricks

Genie + Metric View

- Genie Ontology (2026.6 发布)
- OntoRank 判定定义权威度
- 首答正确率 84.5% vs 通用 52.4%

结论 两家龙头殊途同归: 先有治理良好的 Semantic Layer, 再在其上生长出 Ontology 供 Agent 推理。

于是，两个问题贯穿全场

本次分享围绕它们展开

Q1

如何构建统一的语义层？

在多引擎、多 BI、多方案并存的现实里，怎样得到一套可移植、可治理的指标口径。

→ P6–P13

Q2

从语义层长出的 Ontology 是什么形态？

指标之上的本体，如何组织、如何被 Agent 消费、以什么产品形态落地。

→ P14–P21

Semantic Layer / Model / View: 为什么需要它

在物理表与业务问题之间，架一层可治理的“意义”

Semantic Layer

物理表之上的治理层：把 measures / dimensions / relationships / time 声明式定义一次，成为单一事实源。

让 Agent 面对业务概念，而非裸 schema

Semantic Model

语义层的“代码化”定义：以 YAML 描述实体、join、指标，随项目版本管理 (dbt / MetricFlow)。

偏工程：可 review、可 diff、可复用

Semantic View

语义层的“仓内对象”形态：以 DDL 建在数据库里的一等对象，被 Cortex Analyst / Agent 直接消费。

偏消费：治理与权限随仓库继承

结论 一句话：定义一次，处处一致——让指标与业务逻辑，和具体 dashboard 解耦。

代码侧：git 里的 Semantic Model

Snowflake semantic model YAML · dbt MetricFlow

① 独立引擎 · 随项目版本管理

```
# dbt MetricFlow - semantic model
semantic_models:
- name: orders
  entities:
    - {name: order, type: primary}
    - {name: customer, type: foreign}
  dimensions:
    - {name: order_date, type: time}
  measures:
    - {name: order_total, agg: sum,
      expr: amount}
metrics:
- {name: revenue, type: simple,
  type_params: {measure: order_total}}
```

② 面向 Cortex 消费 · 可转仓内对象

```
# Snowflake semantic model (YAML)
tables:
- name: orders
  primary_key: [order_id]
  dimensions:
    - {name: order_date, expr: ORDER_DATE}
  facts:
    - {name: amount, expr: AMOUNT}
  metrics:
    - name: total_revenue
      expr: SUM(orders.amount)
      synonyms: ['sales', 'turnover']
# → 生成 / 转为 Semantic View
```

结论 同一件事——声明 entities / dimensions / measures / metrics——两种语法、两套生态。

仓内对象侧：数据库里的一等公民

Snowflake Semantic View DDL · Databricks Metric View YAML

① 仓内对象 · 权限随 RBAC 继承

```
-- Snowflake Semantic View (DDL)
CREATE SEMANTIC VIEW sales_sv
  TABLES (orders PRIMARY KEY(order_id),
            customers PRIMARY KEY(cust_id))
  RELATIONSHIPS (
    orders(cust_id) REFERENCES customers)
  FACTS (orders.amount AS order_total)
  DIMENSIONS (customers.region)
  METRICS (
    total_revenue AS SUM(orders.amount)
    WITH SYNONYMS=('sales'));
```

② Metric View (非 semantic view)

```
# Databricks Metric View (Unity Catalog)
version: 1.1
source: samples.tpch.orders
joins:
  - name: customer
    on: source.o_custkey =
        customer.c_custkey
fields: # = 维度
  - {name: mkt, expr: customer.c_mktsegment}
measures:
  - {name: total_revenue,
    expr: SUM(o_totalprice)}
```

结论 注意术语：Databricks 叫 Metric View，用 fields / measures；与 Snowflake Semantic View 同质不同名。

大同小异的语义层

抽象一致，落地各异——这正是孤岛根源

大同 · 共性

- 声明式定义：measures / dimensions
- relationships (join) / time grain
- 定义一次 → 单一事实源，与 dashboard 解耦
- 面向 AI Agent / 自然语言消费
- 目标一致：消除指标漂移，给 LLM 可信 grounding

小异 · 差异 (对比维度)

存储位置： 仓内对象 vs git 代码文件

引擎绑定： 绑定单一平台 vs 引擎独立

表达式方言： 各家 SQL 方言不通用

join / 关系语义： RELATIONSHIPS · cardinality · entities

权限模型： RBAC 继承 vs 语义层自带规则

结论 共性足以标准化，差异足以造成锁定——中间缺的，是一个厂商中立的“最大公约数”。

OSI: 语义层的“最大公约数” Spec

厂商中立、YAML、多方言——define once, query anywhere

是什么

- 2025.9 由 Snowflake 联合 Salesforce/Tableau、dbt Labs 等发起
- v1.0 规范 2026.1 finalized, Apache 2.0, 拟捐 Apache 基金会
- 60+ 成员: Databricks、Cube、AtScale、dbt、BI 与 catalog 厂商
- 核心类: Model / Datasets / Fields / Metrics / Dimensions / Relationships

Converters: OSI ↔ Snowflake • OSI ↔ dbt

```
# OSI - 多方言表达式, 可移植
semantic_model:
  - name: ecommerce_analytics
    datasets:
      - name: orders
        source: sales.public.orders
    metrics:
      - name: total_revenue
        expression:
          dialects:
            - dialect: ANSI_SQL
              expression: SUM(orders.amount)
```

结论 OSI 不取代任何一家, 而是做它们之间的“互换格式”——把 $N \times N$ 的锁定, 降为 N 。

有了统一指标，能做什么

Metric 就是那层统一的接口

Metrics

=

API

=

SQL

=

LLM tools

=

YAML

技术上

- 上卷 Roll-up / 下钻 Drill-down
- 归因 Attribution
- 预测 Forecast
- 物化加速 Materialization

业务上

- 一套北极星指标 / AARRR 体系，引导业务
- 数仓开发 · BI 报表 · AI 问数 · AB 实验 · 人群圈选
- 全部说同一套指标语言

结论 一次定义，处处复用——指标即接口：同一份 YAML，喂给 SQL、BI、Agent 与实验平台。

指标的表达能力，有边界吗？

规范保证基础，能力取决于实现

Custom Extension

OSI 预留 custom extension，供各引擎表达规范外的语义。

跨表粒度对齐 & Join

多表 fan-out、粒度对齐、join 语义，规范未强制统一。

衍生表达方式

复杂窗口、留存 / 漏斗等衍生指标的表达能力各家不同。

关键 能否跨数据源、跨多表、表达复杂窗口与留存/漏斗语义——取决于 Semantic Layer / View 的实现，而非规范。

OSI 基础规范 ≈ 打通 **80%** 基础原子指标互通；**衍生指标 · 跨表语义 · 加速方案** 取决于引擎实现。

结论 OSI 统一“原子指标”的可移植性；衍生表达与加速，正是引擎的竞争区。

Datus OSI Engine

原生 Rust 的 OSI 解析 & 执行引擎

- 原生 Rust 的 OSI parser & 执行引擎
- parser (YAML → SQL) 达 MetricFlow 的 10–20x 性能
- Datus 扩展：Fan-out 正确性、时间表达式语义准确性
- 跨数据源结果一致：

DuckDB

StarRocks

ClickHouse

Doris

TiDB

Trino

Postgres

MySQL

Snowflake

BigQuery

Databricks

parser 性能

10–20x

YAML → SQL, 对比 MetricFlow

语义正确 + 方言一致，是引擎的护城河

OSI YAML
osi-model

Semantic IR
osi-compiler

Logical Plan
osi-planner

SQL AST
polyglot-sql

Dialect SQL

什么是本体 (Ontology) ?

从“描述世界”，到“驱动业务操作”的演进

① 语义网时代

RDFS / OWL

- 用 classes / properties / relationships 形式化“描述世界”
- 面向知识表示与逻辑推理
- 定位：一张静态的“世界知识图”



② Palantir 之后

Ontology = 组织的 digital twin / operational layer

- datasets / models → Object Types · Properties · Links
- 再加入 Actions · Functions · 安全机制
- 定位：从“描述世界” → “驱动业务操作”

结论 本场要的正正是第二种：可被 Agent 消费、能驱动操作的本体——长在语义层与指标之上。

语义层之上，如何“长出”本体

以 Snowflake Ontology 为例：用表建图，KG_NODE + KG_EDGE

本体 = 语义层之上的一层意义

- 语义层给出实体 / 指标 / join；本体再叠层级、同义、领域关系
- 用 Snowflake 表直接建图，无需独立图数据库：

KG_NODE 实体：id / type / 属性 (VARIANT)

KG_EDGE 关系：src / dst / type / 元数据

```
-- Recursive CTE: 动态多跳遍历层级
WITH RECURSIVE paths AS (
  SELECT src, dst, 1 AS depth
  FROM edges WHERE src='EpithelialCell'
  UNION ALL
  SELECT p.src, e.dst, p.depth+1
  FROM paths p JOIN edges e
    ON p.dst = e.src)
SELECT * FROM paths;
-- subClassOf 可变长展开，数据驱动
```

结论 本体不必外挂图数据库：节点边两张表 + 递归 CTE，就能在仓内做层级 / 传递 / 同义推理。

路线一：Knowledge Graph + Semantic Model

直接出 SQL · 更探索 (Snowflake KG 查询流程)



特点

- 运行时遍历图，动态展开层级；确定性强、覆盖深（如 693 个后代跨 10+ 层）
- 共 7 个工具（4 存储过程 + 2 语义视图 + 基线）：编排复杂、需精确概念名，无同义解析
- 灵活但方差大——能力最强，也最考验 Agent 的工具选择

结论 KG 路线：把本体当图来遍历——最灵活，但工具多、方差高。

路线二：Flattened GraphRAG + Semantic View

直接用指标 · 更稳定 (预计算画像 + 检索)

① 建索引



② 查询 (仅 2 工具)



Benchmark (Snowflake 生物学 22 题, 首答成功率)



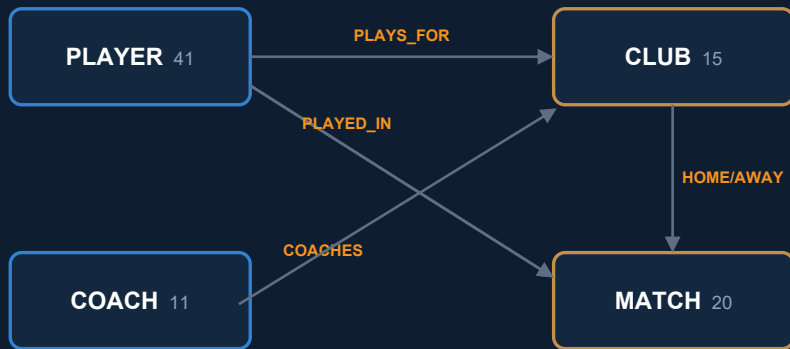
结论 GraphRAG 路线: 预计算画像 + 两工具检索——同义解析强、方差最低, 78.2% 最优。

本体构建示例：Snowflake 足球知识图谱

Snowflake-Labs / ontology-on-snowflake · 同一份点边，两层视角

场景 足球领域：球员 · 教练 · 俱乐部 · 比赛 · 合同（15 俱乐部 / 41 球员 / 11 教练 / 20 比赛）；元数据驱动，对标 Palantir

具体层 · Concrete KG (KG_NODE + KG_EDGE)



抽象层 · Ontology (VW_ONT_*)



WORKS_FOR = PLAYS_FOR U COACHES

PARTICIPATES_IN = PLAYED_IN U HOME/AWAY_TEAM

同一批点边 → 抽象归并，跨类型推理

结论 同一份 KG_NODE/KG_EDGE：具体层答“谁效力皇马”，抽象层答“皇马所有相关人员（球员+教练）”。

给 Agent 配置的 8 个工具 (3 + 3 + 2)

A 类 · Cortex Analyst text-to-SQL (自然语言 → SQL, 绑定 Semantic Model YAML)

- | | |
|-----------------------------|--------------------------------------|
| 1 query_soccer_data | soccer_semantic_model.yml |
| 2 query_soccer_ontology | soccer_semantic_model_ontology.yml |
| 3 query_metadata_governance | soccer_metadata_governance_model.yml |

B 类 · 图分析 (generic → SPCS Service Function · NetworkX)

- | | | |
|--|---|---|
| 4 centrality_tool
/centrality · 中心性 | 5 community_detection_tool
/community-detect · Louvain | 6 shortest_path_tool
/shortest-path · 最短路径 |
|--|---|---|

C 类 · pandas 程序补充工具

- | | |
|---|--|
| 7 temporal_analysis_tool
/temporal-analysis · 时序演化 | 8 transfer_analysis_tool
/transfer-network · 转会网络 |
|---|--|

A 类接 Semantic Model; B 图算法&图遍历 C 程序补充工具

OSI 0.2.0: Ontology Preview

Apache Ossie · v0.2.0.dev0 (2026-05) : 在指标之上, 声明本体

Concepts

EntityType (实体) / ValueType (值类型) ; 内建
Any · Integer · String · Date ...; extends 继承

Relationships

roles + multiplicity (ManyToOne / OneToOne) ; verbalizes 自然语言
模板供 LLM 消费

Business rules

derived_by (派生 / 递归, 如 ancestor_of) + requires (约束)

Ontology mappings

逻辑字段 → 概念 / 关系: object · link · referent mappings

```
# Apache Ossie - ontology (0.2.0)
name: EnterpriseOntology
ontology:
  - concept:
      name: Person
      type: EntityType
  relationships:
    - name: earns
      roles: [{concept: Salary}]
      multiplicity: ManyToOne
      verbalizes:
        - '{Person} earns {Salary}'
```

结论 指标统一“怎么算”，本体统一“是什么、怎么连、怎么说”——verbalizes 让 Agent 拿到可验证语义。

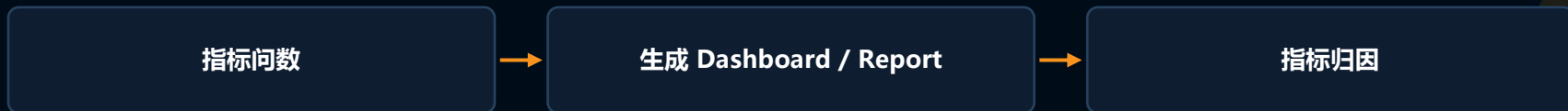
有了指标 + 本体，能做什么

从看数、问数，到本体驱动的洞察与行动

数据开发闭环



指标消费闭环



本体洞察层 · Insight & Action



结论 本体站在指标之上：不止问数看数，而是从更高层给出数据驱动的行动建议与洞察。

如何初始化指标体系

两种视角对齐，三条来源汇入

业务导向

需要哪些 Semantic View / Metrics——从“要回答什么业务问题”出发

工程导向

需要哪些 Semantic Model / key filter & relation——从“数据怎么连”出发

1 从历史 SQL 抽取

挖掘聚合模式 → 候选指标，带置信度与去重

2 从历史 Dashboard 抽取

从既有 BI 报表反解出指标定义

3 从开发中新建

在数据建设流程里顺带沉淀指标

结论 指标不是凭空规划出来的，而是从历史 SQL、Dashboard 与开发产物里“抽”与“长”出来的。

Top-down 为什么失败

两道坎，最终沦为没人用的指标目录

POLITICAL

政治

谁说了算？口径归属、优先级、责任边界谈不拢——统一还没产生价值，就先陷入拉锯。

TECHNICAL

技术

生产端与消费端打不通：定义在一处，查询在另一处，指标目录与真实 SQL / Job 各行其是。

结局 一个漂亮但没人用的指标目录——不是不需要统一，而是想在产生价值前先统一。

像管代码一样管 SQL / Job / Metrics

Data Governance for Agent

版本

Version

PR

Pull Request

Review

评审

Merge

合并

血缘

Lineage

为什么这样管才对

指标是被反复查询、验证的工程产物——它天然适合用软件工程的方式治理。

把指标纳入版本 / PR / review / merge / 血缘，Agent 消费的每一个口径都可追溯、可回滚、可审计。

结论 治理的对象不再是静态目录，而是活的工程产物——这就是面向 Agent 的数据治理。

Datus Semantic Hub: 从个人指标, 到公司资产

像 GitHub 一样, push / pull + PR review 逐级晋升



结论 容许不一致: 先在 project / workspace 交付价值, 再经 PR 逐步收敛, 晋升为公司级指标资产。

Bottom-up 路径：先交付，再统一

从数据工程的真实产物里“长”出指标



为什么 Bottom-up 成立

企业级 ontology 不是手工画出来的，而是从被反复查询、验证的 metrics 中“长”出来的。
Metric 是 data ontology 的基石——先让指标在真实使用中被检验，再渐进统一到 enterprise。

结论 从 top-down 治理，到 bottom-up 演化——AI Native、开放不绑定（OSI）、容许不一致。

三个 Takeaway

1

语义层是 Agent 的契约语言

准确、可治理的指标 / 知识库 / 本体，比更大的模型更能让 Agent 稳定回答业务问题。

2

开放胜过孤岛：OSI

厂商中立的最大公约数 spec，把 $N \times N$ 的语义锁定降为 N ，让指标 define once、query anywhere。

3

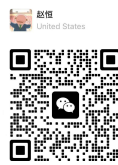
Bottom-up 胜过 Top-down

像管代码一样管指标，从真实产物里长出本体，先交付价值再渐进统一。



Star Datus-agent
+ 报名 POC

github.com/Datus-ai/Datus-agent



扫码加我个人微信

赵恒 @datus.ai

谢谢 • THANKS